

Project 1 – ML Pitfalls

1. As we discussed in lecture, NNs (and most ML systems) can approximate almost any function. While this is powerful, it is also dangerous. I’ve prepared a pair of data sets that are a toy model for two-particle decay through two different channels:
 - (a) There are two latent variables – modeled by Gaussian distributions – representing the two decay channels. Each variable has their own mean and standard deviation.
 - (b) One of the two latent variables is chosen (with equal weight), and used as the center of another normal distribution with $\sigma = 1$ – representing the detector interaction. Two points are picked from this distribution to produce the “measured energy” of the decay as the “data” vector.
 - (c) Each data point is labeled based on which of the two latent variables it was drawn from.

One of the two data sets has very minor but specific post-processing, as might be induced by any number of incidental hardware processing steps. The data sets are available on Canvas via the `wz_data#.npz` and `wz_labels#.npz` files in Project 1.

Additionally, I have provided some helper functions, including a simple classifier model, training loop, and validation loop in the Project 1 repo on GitLab.

Train, validate, and test the model using each of the two data sets (a separate model trained on each set).

- Which of the models do you think is “better”? Why? What criteria did you use for “better”?
 - Which of the data sets has had the post-processing?
2. Read *Is machine learning good or bad for the natural sciences?* [arxiv:2405.18095]. We will center our discussion around it on Monday. Then work through the example provided in Appendix A:
 - (a) I have provided simulated datasets (a **labeled training set** and an **unlabeled test set**) on Canvas and a **PyTorch MLP regression classifier (and training loop)** on GitLab. If you are interested, you can also nearly-trivially set up an MLPRegression classifier using SciKit Learn (most of the defaults are reasonable; you just need to set the hidden layer structure, the activation function – ReLU – and maybe fiddle with the optimizer – **adam** is a common well-balanced choice).
 - (b) **Build a histogram comparing the true y to the guiding radius for the training and validation sets.**
 - (c) **Train the classifier on the training data.** Feel free to **tweak some of the training hyperparameters** until you are “happy” with the model training.
 - (d) **Check your classifier on the validation data set.** Does it appear to have **“learned” anything?** Is it producing a reasonable distribution of classifications?
 - (e) **Build a histogram comparing the estimated $\hat{y}$ to the guiding radius** for the training and validation sets.
 - (f) **Run your classifier on the test (unlabeled) data set, and build a similar histogram comparing these predicted/estimated $\hat{y}$ to their guiding radius.**
 - (g) **Compare the three histograms** (the true labels, the estimation on the known set, and the estimation on the test set). How does the estimation of the test set compare with the “ground truth”?
 - (h) If you naively computed the standard error of each histogram bin (for each histogram), which appears to have the smallest error? In terms of the test set’s statistics, what is it’s worst deviation from the truth?